MagicQ miniQ ArtNet-USB

This handbook describes a **4-port Art-Net <-> DMX bridge** built around a Raspberry Pi Pico (RP2040). The Pico appears to your stage-lighting PC (MagicQ & co.) as a **USB network adapter**, exchanges Art-Net over that USB link, and drives four DMX ports on GPIO2, GPIO3, GPIO4 and GPIO5.

Each of the four ports is individually configurable as an **output** (ArtDmx -> DMX out) or an **input** (DMX in -> ArtDmx), so the same node can drive a rig, read a rig, or do both at once. On top of the DMX ports the node answers Art-Net discovery (ArtPoll/ArtPollReply), accepts addressing over the network (ArtAddress), bridges RDM, and serves a small config web page.

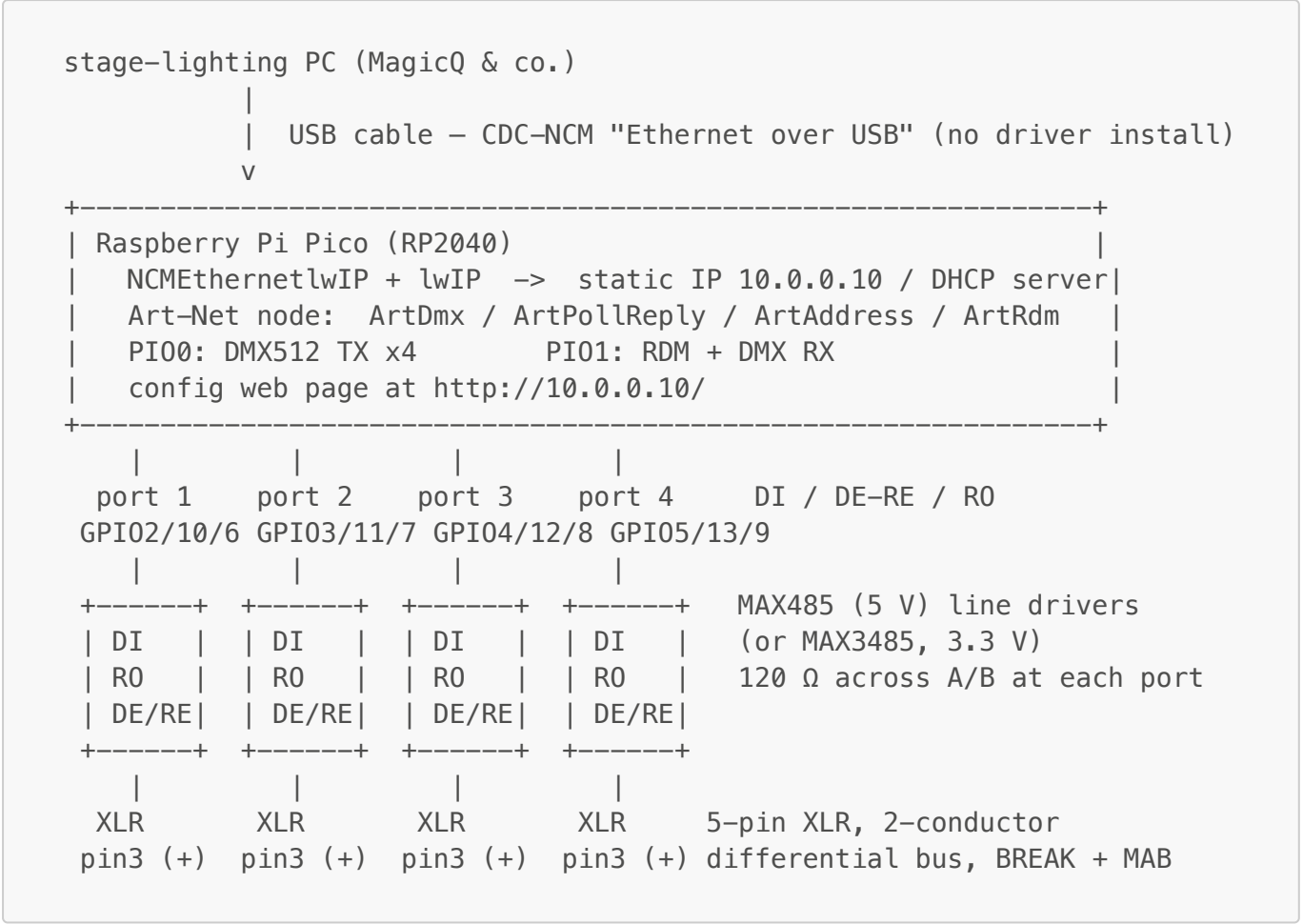
> **Note:** The Pico has no Wi-Fi or wired Ethernet of its own — the USB link *is* the network. Modern Windows 10/11, macOS and Linux mount an NCM network adapter with **no driver install** (see section 1.1).

Table of contents

- 1. How it works
  - 1.1 USB networking: Ethernet over the native USB port
  - 1.2 Addressing: static IP or DHCP server
  - 1.3 DMX and RDM: the PIO state machines
  - 1.4 Firmware responsibilities in `src/main.cpp`
- 2. Required hardware
- 3. Hardware wiring
  - 3.1 DMX port ↔ Pico pin assignment
  - 3.2 Transceivers (MAX485 vs MAX3485)
  - 3.3 Auto-direction transceivers ("automatic flow control")
  - 3.4 Pico ↔ MAX485 signal levels
  - 3.5 Termination
  - 3.6 DMX input mode
- 4. Build & flash
  - 4.1 Build with PlatformIO
  - 4.2 Copy the UF2 manually
  - 4.3 Upload with a debug probe / PIO
  - 4.4 Pin the exact platform/version (recommended)
- 5. Setup on the host PC
  - 5.1 The USB network adapter & addressing
  - 5.2 Art-Net output in MagicQ
  - 5.3 The config web page
  - 5.4 Remote programming via ArtAddress
  - 5.5 macOS: the NCM interface
- 6. Notes & hardware verification
- Project structure

1. How it works

Art-Net arrives over USB, is parsed in firmware and leaves the node as DMX512 on four RS485 ports; DMX input ports work the other way round and publish what they receive back as ArtDmx:



1.1 USB networking: Ethernet over the native USB port

- **USB networking**: the RP2040's native USB runs the **CDC-NCM** "Ethernet over USB" class via the Arduino-Pico core's `NCMEthernetlwIP` (TinyUSB + lwIP). Modern Windows 10/11, macOS and Linux mount an NCM network adapter with **no driver install**. The Pico has no Wi-Fi or wired Ethernet of its own — the USB link *is* the network. See *macOS: the NCM interface* below for two firmware details that keep the link reliable on Apple machines.

1.2 Addressing: static IP or DHCP server

- The device gets a **static IP 10.0.0.10** on the USB link by default, and can run a **DHCP server** that hands the host an automatic address in **10.0.0.1–10.0.0.9** — no manual IP configuration needed on the PC. IP, netmask and the DHCP server are configurable on the embedded web page (http://10.0.0.10/).

1.3 DMX and RDM: the PIO state machines

- **DMX**: a custom `DmxOut` library (`lib/DmxOut/`) drives each output with a PIO state machine + DMA. Four universes on GPIO 2/3/4/5, one SM per universe on `pio0`. It emits spec-compliant framing for strict receivers: BREAK ~185 µs, MAB 16 µs, 250 kbaud data bits (the bundled Pico-DMX's 8 µs MAB was rejected by some fixtures).

GPIO numbers match the Arduino pin numbers on the plain Pico. RDM RX runs on `pio1` (one state machine per port); the DMX outputs continue to use `pio0`.

DMX/RDM share the same 2-conductor differential bus over a 5-pin XLR. RDM is half-duplex at 250 kbaud with a BREAK + MAB frame; the gateway relays Art-RDM commands from MagicQ to the fixtures and bridges their responses back.

> **Note:** MagicQ declares a short ArtDmx length but always sends 512 slots. The node uses the real datagram size, so all channels pass through.

1.4 Firmware responsibilities in `src/main.cpp`

- DMX driver (DmxOut, PIO + DMA)
- USB CDC-NCM Ethernet (NCMEthernetlwIP + lwIP, static IP 10.0.0.10)
- DHCP server for the USB link (10.0.0.1-9)
- Art-Net OpDmx parser over a raw UDP socket (no external Art-Net lib)
- ArtPoll/ArtPollReply responder -> MagicQ auto-discovers all 4 universes
- ArtAddress (0x6000) responder -> MagicQ can remap net/subnet/universe and switch a port's direction over Art-Net; confirmed with a fresh unicast ArtPollReply
- DMX input ports publish received frames as ArtDmx broadcasts
- Config web page at http://10.0.0.10/ (WebServer; per-port direction + net/subnet/universe)

2. Required hardware

- **1x Raspberry Pi Pico** (RP2040, plain Pico — the GPIO numbers used in this handbook match its Arduino pin numbering).
- **4x MAX485 (5 V)** RS485 transceivers — one half-duplex transceiver per DMX port, the recommended line driver.
- **4x MAX3485 (3.3 V)** — an *alternative* to the MAX485s, not an addition; can run off 3.3 V, but the lower bus voltage is rejected by some fixtures.
- **4x 120 Ω resistors** — one termination per port, across A and B at the transceiver.
- **4x 10 kΩ resistors** — one series resistor per port, in the `RO` line between the transceiver and the Pico GPIO.

> **Important:** Quantities above follow the **4-port** design documented here. Details, rationale and alternatives are in section 3; never invent a parts list beyond what the wiring sections name.

3. Hardware wiring

3.1 DMX port ↔ Pico pin assignment

DMX port	Pico GPIO	MAX485 signal
Universe 1	GPIO2 (`DI`) / GPIO10 (`RO`, via 10 kΩ) / GPIO6 (`DE`/`RE`)	MAX485 DI / RO / direction
Universe 2	GPIO3 (`DI`) / GPIO11 (`RO`, via 10 kΩ) / GPIO7 (`DE`/`RE`)	MAX485 DI / RO / direction
Universe 3	GPIO4 (`DI`) / GPIO12 (`RO`, via 10 kΩ) / GPIO8 (`DE`/`RE`)	MAX485 DI / RO / direction
Universe 4	GPIO5 (`DI`) / GPIO13 (`RO`, via 10 kΩ) / GPIO9 (`DE`/`RE`)	MAX485 DI / RO / direction

3.2 Transceivers (MAX485 vs MAX3485)

Each port uses a half-duplex RS485 transceiver:

- `DI` → Pico GPIO (TX): DMX **and** RDM transmit data.
- `RO` → Pico GPIO (RX): RDM receive data; on a port configured as an **input** this is the DMX receive path and must be wired (via the 10 kΩ series resistor). Not needed on an output-only port.
- `DE`/`RE` → Pico GPIO (direction): driven high to transmit, low to receive. These are toggled automatically when RDM is used; on an **input** port `DE` is held low permanently so the receiver is always enabled.

Two options for the line driver:

- **MAX485 (5 V) — recommended for picky fixtures.** `VCC` = 5 V from VBUS (Pico pin 40); bus levels swing to ~5 V differential. Its logic side is TTL, so the 3.3 V GPIOs drive `DI`/`DE` directly; `RO` is 5 V into a non-tolerant GPIO, handled with a 10 kΩ series resistor (see below).
- **MAX3485 (3.3 V).** Simplest wiring (can run off 3.3 V), but the lower bus voltage (~2.5 V) is rejected by some fixtures.

3.3 Auto-direction transceivers ("automatic flow control")

Auto-direction RS485 modules (e.g. MAX13487E-class) derive the direction from the data line instead of a `DE`/`RE` input, saving those wires. Assessment for this node:

- **DMX output only**: mostly works, but the driver goes high-impedance during the idle *mark* (TXD high). DMX has no bias resistors by design, so the ~60 Ω combined termination cannot reliably hold a valid mark → some fixtures glitch or drop out. If you must use one, add bias resistors (pull-up on A, pull-down on B) near the transceiver and re-verify with your fixtures (the node's picky 4-ch par is a good test case).
- **RDM (all 4 ports here)**: not recommended. RDM needs the bus actively driven through BREAK/MAB and the ~11-bit response window, plus a clean bidirectional handoff — exactly what a real `DE`/`RE` pin provides. Auto-flow cannot do this reliably.

So keep the `DE`/`RE` wiring for all 4 ports if RDM stays enabled; drop the direction pins only if a port is made DMX-only and you accept the bias caveat.

> **Important:** With RDM enabled on all 4 ports, the `DE`/`RE` wiring is not optional — an auto-direction module cannot provide the clean bidirectional handoff RDM needs.

3.4 Pico ↔ MAX485 signal levels

The Pico IOs are 3.3 V and **not 5 V tolerant**. The MAX485's logic side is TTL:

- **`DI` (TX) and `DE`/`RE` (direction)** are driven **directly** by the 3.3 V GPIOs — TTL inputs switch at ~2.0 V, so a 3.3 V high is guaranteed valid. No level shifting needed.
- **`RO` (RX)** is a push-pull output swinging 0–5 V. Feed it into the GPIO through a **10 kΩ series resistor**: the Pico's internal ESD clamp limits the line to ~3.6 V while the 10 kΩ keeps the clamp current at ~170 µA (safe), and the resulting RC (10 kΩ into the sub-20 pF input) settles in ~200 ns — far shorter than the 4 µs RDM bit time, so 250 kbaud reception is unaffected.

> **Note:** A TXS0108-based alternative was considered for full 5 V→3.3 V level shifting on all eight signals; the direct `DI`/`DE` + 10 kΩ-on-`RO` wiring above replaces it.

3.5 Termination

A 120 Ω resistor **across A and B** (differential, not in series with A) at each port's transceiver — one per port:

```
A (pin7) ──────────────── XLR pin3 (+)
           |
         120 Ω
           |
B (pin6) ──────────────── XLR pin2 (−)
```

If the far-end fixture already terminates (usual), the two in parallel load to ~60 Ω — fine for short runs; drop the node-side one if a fixture misbehaves.

3.6 DMX input mode

A port configured as an **input** disables its DMX transmitter (pico drives `DE` low) and listens on `RO`. The same `pio1` receiver used for RDM captures the incoming DMX stream; a small state machine resynchronises on BREAK, checks the `0x00` start code, collects the 512 slots and broadcasts the frame as ArtDmx on the port's configured `net`/`subnet`/`universe`. Input ports ignore incoming ArtDmx and do not bridge RDM — they only publish frames. An input port with signal within the last second is advertised in ArtPollReply with a DMX-input PortType, `GoodInput` "data received" and its `SwIn` set, so controllers can discover it.

4. Build & flash

4.1 Build with PlatformIO

```
pio run              # build -> .pio/build/pico/firmware.uf2
```

4.2 Copy the UF2 manually

Or just build and copy the UF2 manually:

1. Hold the **BOOTSEL** button on the Pico, plug in USB, and release.
2. A mass-storage drive named `RPI-RP2` appears. Copy `.pio/build/pico/firmware.uf2` onto it. The Pico reboots and runs the firmware.

4.3 Upload with a debug probe / PIO

Alternatively with a debug probe / PIO upload:

```
pio run -t upload    # needs a UF2/OpenOCD/dbgprobe upload_protocol
configured
```

4.4 Pin the exact platform/version (recommended)

`platform = https://github.com/maxgerhardt/platform-raspberrypi.git` tracks the latest. For a reproducible build pin a specific tag, e.g.:

```
platform = https://github.com/maxgerhardt/platform-raspberrypi.git#1.10.0
```

5. Setup on the host PC

5.1 The USB network adapter & addressing

1. Connect the Pico to the PC with a USB cable.
2. The PC enumerates a "USB Ethernet / RNDIS/NCM" adapter.
3. **Nothing to configure** — the Pico's DHCP server gives the adapter an address in **10.0.0.1–10.0.0.9** automatically (just ensure the adapter is set to "Obtain an IP address automatically", the default). The Pico answers on **10.0.0.10**.

5.2 Art-Net output in MagicQ

1. In MagicQ, add an Art-Net output and point it at **10.0.0.10**. The node answers ArtPoll, so MagicQ auto-discovers the ports — no manual mapping needed (see the config page for remapping universes).

5.3 The config web page

Open **http://10.0.0.10/** in a browser (host is on the USB link, or any machine that can route to the node's IP). The page follows the classic node layout with two tabs:

- **Channels** — live view of all 512 DMX channel values of the selected port (intensity grid, search, "active only" filter). Polls `/api/status` ~3×/s.
- **Settings** — configure and store (flash/EEPROM):
  - **Network**: static IP, netmask, and an on/off switch for the DHCP server.
  - **Art-Net ports**: for each of the 4 DMX ports choose the **direction** (`output` = ArtDmx -> DMX out, `input` = DMX in -> ArtDmx) and set the full Art-Net address, `net` (0–127), `subnet` (0–15), `universe` (0–15). The 15-bit address a port uses is `net<<8 | subnet<<4 | universe`; it applies to ArtDmx data (and ArtRdm traffic for output ports).
  - **Node identity**: **Short name** (max 17 chars) and **Long name** (max 63 chars) advertised in the ArtPollReply. MagicQ shows them in its output node list; the short name also appears in the web header pill. Both are stored in flash alongside the network settings.
  - **Save & reboot** applies everything (node reboots, then re-announces via ArtPollReply so MagicQ picks up the new mapping).

All settings — direction, per-port address, IP/netmask/DHCP, node names — are stored in flash (EEPROM, layout v5) and survive power cycles and reboots.

A save triggers a warm reboot. Before resetting, the node intentionally drops the USB link so the host always sees the device unplug and re-enumerates the NCM interface; this takes a few seconds. If the config page does not return immediately, give the host a moment to re-run DHCP, then reopen `http://<new-ip>/`.

The same API the page uses: `GET /api/status?port=N`, `GET/POST /api/config`, `POST /api/reboot`, `POST /api/factory`. `/api/config` carries the address triples as arrays `net`, `subnet`, `universe` plus a `direction` array (0 = output, 1 = input) and the node names `name`/`longname`; the POST form uses fields `n0..n3`, `s0..s3`, `u0..u3`, `d0..d3`, `name` and `longname`. `/api/status` also returns `direction`, a per-port `input_active` flag and the active `name`.

Defaults:

Setting	Default
IP	`10.0.0.10`
Netmask	`255.0.0.0`
DHCP server	**on** (pool `<ip>.1–.9`)
Port direction (all four)	**output**
Port addresses (net/sub/universe)	`0/0/0`, `0/0/1`, `0/0/2`, `0/0/3`

If you change the subnet or disable DHCP, either reach the new IP from a manually-configured host, or hold BOOTSEL and re-flash the factory firmware.

MagicQ mapping with the defaults: MagicQ Uni 1 -> address 0/0/0 -> DMX GPIO2, Uni 2 -> 0/0/1 -> GPIO3, Uni 3 -> 0/0/2 -> GPIO4, Uni 4 -> 0/0/3 -> GPIO5. The node advertises `SwOut = subnet<<4|universe` per port (exact for net 0, the typical setup).

5.4 Remote programming via ArtAddress

Instead of the web page, a controller (MagicQ) can reprogram addressing and port direction over the network by sending an **ArtAddress** packet (OpCode `0x6000`) directly to the node's IP. The node parses it, applies the changes live, stores them in flash, and confirms by unicasting a **fresh ArtPollReply** that reflects the new state (so MagicQ picks up the new SwIn/SwOut mappings automatically).

Supported ArtAddress fields:

Field	Byte	Effect
NetSwitch	12	New `net` (0–127), applied to all ports when the new-value flag (bit 7) is set; `0x7F` = no change
ShortName	14–31	Node short name (ignored if the string is null)
LongName	32–95	Node long name (ignored if the string is null)

Field	Byte	Effect
SwIn[4]	96–99	Universe (0–15) for **input** ports, applied when bit 7 is set
SwOut[4]	100–103	Universe (0–15) for **output** ports, applied when bit 7 is set
SubSwitch	104	New `subnet` (0–15) for all ports, applied when bit 7 is set
Command	106	`0x20–0x23` = set port 0–3 to **output**, `0x30–0x33` = set port 0–3 to **input**

A port switched to **input** immediately stops streaming DMX and `DE` is held low so the receiver listens on the bus; its `DMX in -> ArtDmx` path starts publishing frames on the configured address. A port switched to **output** resumes streaming `ArtDmx -> DMX` (and RDM bridging). Both direction changes persist across power cycles.

5.5 macOS: the NCM interface

Two firmware details, both in the vendored `lib/lwIP_USB_NCM/` plus `src/main.cpp`, keep the Ethernet-over-USB link reliable on macOS (Apple's `com.apple.driver.usb.cdc.ncm`):

1. **ncm_bmNetworkCapabilities = 0x01.** Without it macOS enumerates the NCM interface but never brings the link up; advertising `SetEthernetPacketFilter` in the NCM functional descriptor (`desc[45] = 0x01` in `usbInterfaceCB`) works around Apple's bug (r.170072016). Windows/Linux are unaffected but accept the field.

2. **NCM registration must happen *after* the `USBClass` constructor.** The RP2040 core's `src/main.cpp` runs all static constructors, then calls `USB.begin()` which *resets* the interface/endpoint/string tables. If `NCMEthernet` registers itself from its own constructor, that registration is silently wiped before the device enumerates — with only CDC/Serial surviving. Symptom: no NCM interface on the host, `enX` never appears. Fix: the ctor is empty and `src/main.cpp` calls `eth.usbRegisterInterfaces()` from the framework's `initVariant()` hook (runs after all static ctors, before `USB.begin()`).

A healthy macOS boot shows a CDC ACM interface (`Pico Serial`), a CDC data interface, an NCM control interface (`Pico NCM`, interface class 2 sub-class 13) and an NCM data interface; the host then creates an `enX` (e.g. `en7`) with an address from the Pico's DHCP pool (`10.0.0.1`). `ping 10.0.0.10` and the config page are the quick checks.

6. Notes & hardware verification

- **Serial logs**: with USB used for NCM networking, the standard `Serial` USB-CDC and an NCM interface can share the USB port. In the earlephilhower core `Serial` (USB CDC) works alongside the NCM class. If you prefer a hardware UART, use `Serial1` on the UART0 pins (GPIO0 TX / GPIO1 RX) after `Serial1.begin(...)`.
- The NCM link is confirmed on the Pico side via `eth.linkStatus() == LinkON` (i.e. the host has mounted the USB NCM interface). The node only reports `READY` once that is true; if the host is slow to enumerate it keeps checking and recovers without a re-plug. If `ping 10.0.0.10` fails, first check that the host has picked up the NCM adapter and is on the 10.0.0.x subnet (on macOS, see *macOS: the NCM interface* above).

- **Warm reboot / Save & reboot**: a plain watchdog reset can leave the USB pull-up asserted, so the host never notices the disconnect and fails to re-enumerate the NCM interface (the page then only works after a cold boot). `rebootNode()` therefore calls `USB.disconnect()` (blocking ~500 ms) before resetting, and `bringUpNetwork()` waits for the host to re-enumerate.
- **Testing**: to verify without a lighting rig, drive an LED from one universe's output, or scope the RS485 TX line — a DMX output idles high after each frame.

Project structure

```
ArtNet-USB/
├── README.md            # This handbook
├── platformio.ini       # Build config (RP2040, Arduino-Pico /
earlephilhower core)
├── src/
│   ├── main.cpp         # Node firmware (DMX, NCM Ethernet, Art-Net, RDM,
web page)
│   └── index_html.h     # Embedded config web page
└── lib/
    ├── DmxOut/          # Custom DMX512 TX PIO (BREAK ~185 µs, MAB 16 µs)
+ DMA driver
    ├── ArtnetRdm/       # RDM gateway: E1.20 protocol, per-port RDM RX
(pio1), ArtRdm bridge
    ├── ArtnetConfig/    # Persisted per-port direction + Art-Net address
map (EEPROM-backed)
    └── lwIP_USB_NCM/    # Vendored NCMEthernetlwIP with the local fixes
(section 5.5)
```

> The Art-Net PCB design sources are kept in the shared `../easyEDA/` project.

What the sources contribute:

```
platformio.ini      build config (RP2040, Arduino-Pico / earlephilhower
core)
lib/DmxOut/          custom DMX512 TX PIO (BREAK ~185 µs, MAB 16 µs) + DMA
driver
lib/ArtnetRdm/       RDM gateway: E1.20 protocol, per-port RDM RX (pio1),
ArtRdm bridge
lib/ArtnetConfig/    persisted per-port direction + Art-Net address map
(EEPROM-backed)
lib/lwIP_USB_NCM/    vendored NCMEthernetlwIP with the local fixes below
src/main.cpp
    - DMX driver (DmxOut, PIO + DMA)
    - USB CDC-NCM Ethernet (NCMEthernetlwIP + lwIP, static IP 10.0.0.10)
    - DHCP server for the USB link (10.0.0.1-9)
    - Art-Net OpDmx parser over a raw UDP socket (no external Art-Net lib)
    - ArtPoll/ArtPollReply responder -> MagicQ auto-discovers all 4
universes
      (note: MagicQ declares a short ArtDmx length but always sends 512
slots;
```

```
      the node uses the real datagram size, so all channels pass through)
    - ArtAddress (0x6000) responder -> MagicQ can remap net/subnet/universe
and
      switch a port's direction over Art-Net; confirmed with a fresh
unicast ArtPollReply
    - DMX input ports publish received frames as ArtDmx broadcasts
    - Config web page at http://10.0.0.10/ (WebServer; per-port direction +
net/subnet/universe)
```